Espressioni, tipi, e il modello di riduzione

L03 · Programmazione funzionale

Mauro Leoncini

Obiettivi

- Espressioni vs comandi; `let` come **legame** e non come assegnamento
- **Trasparenza referenziale**, cosa è e a che cosa serve
- **Inferenza di tipo**
- **Il modello di riduzione**: valori, redex, chiamata per **valore**, chiamata per **nome**

Da L02: che cosa la frase non diceva

> Data una lista di stringhe, restituisci la più lunga.

I casi che l'italiano lascia aperti.

- due stringhe **di pari lunghezza massima** — quale delle due?
- la lista **vuota** — che cosa restituite?
- la stringa vuota, se è l'unica

Qualsiasi scelta **purché meditata** non è errata: la frase non decideva, e la decisione spetta a chi scrive la specifica.

Il tipo "dice" già qualcosa

Stabilendo la firma della funzione, ad esempio:

```
piu_lunga : string list -> string
```

stiamo già dicendo qualcosa di più preciso!

Infatti, questa firma **promette** sempre una stringa ma sulla lista vuota (e eventualmente anche nel caso di stringhe di uguale lunghezza) non c'è niente da restituire. La promessa è impossibile, e si vede prima di scrivere il corpo.

> Il tipo è la prima specifica, ed è l'unica che "qualcuno" controlla per noi.

Una cosa che la proprietà dava per scontata

Le proprietà che avete scritto presumibilmente hanno questa forma:

```
per ogni s nella lista,  lunghezza s <= lunghezza (piu_lunga lista)
```

Perché questa frase abbia senso, `piu_lunga lista` deve **denotare un valore**, sempre lo stesso, indipendentemente da quando la si guarda.

In C++ quella stessa frase può diventare falsa un istante dopo, senza che nessuno abbia richiamato la funzione: basta che "qualcuno" (codice concorrente) abbia un riferimento alla lista

Che cosa rende lecita quella scrittura?

`let` non è assegnamento

Veniamo ora agli argomenti propri di questa lezione.

In C++ scriviamo **comandi**, che si eseguono in ordine e lasciano tracce (nello stato):

```
int x;
x = 3;
x = x + 1;
```

In OCaml scriviamo **espressioni**, che si riducono a un valore:

```
let x = 3 in x + 1
```

Non c'è un «prima» e un «dopo». C'è un'espressione e il suo valore.

`let` non è assegnamento

```
let x = 3
let x = x + 1
```

La seconda riga **non modifica** `x`: introduce un nuovo legame che nasconde il precedente. Il vecchio `x` non è cambiato, è solo diventato irraggiungibile per nome.

La dimostrazione che è un legame

```
let x = 3
let f () = x + 1
let y = f ()
let x = 100
let z = f ()
```

Quanto vale `y`? e `z`?

Sia `y` che `z` valgono **4**, non 101.

`f` ha catturato il legame che esisteva quando è stata definita. Se `let` fosse assegnamento, `f ()` varrebbe 101.

Trasparenza referenziale

Un'espressione è **referenzialmente trasparente** quando può essere sostituita dal proprio valore in qualunque contesto, senza cambiare il significato del programma.

Si tratta di una sorta di **permesso**: la trasperenza referenziale ci autorizza a ragionare sulle proprietà di un programma **per sostituzione**, cioè a fare con i programmi quello che si fa in Matematica con le equazioni.

Più spesso la trasparenza referenziale si trova definita come la proprietà di non avere *effetti collaterali* (*side effect*), ma la nostra definizione implica questa.

Dove il permesso non c'è

Riconsideriamo l'esempio C++, inserendolo in un contesto che lo renda **compilabile**:

```
int x = 3;
int f() { return x + 1; }
int main() {
    std::cout << f() << std::endl;   /* Stampa 4 */
    x = 100;
    std::cout << f() << std::endl;   /* Stampa 101 */
```

La mera esistenza degli assegnamenti semplicemente rende impossibile la trasparenza referenziale.

Non esiste un valore univoco associato all'espressione `f()`

Dove il permesso non c'è

Altri due controesempi che possiamo trovare in un programma Python:

```
random.random()       # due occorrenze, due valori
l.append(1)           # il valore restituito e' None,
                      # ma qualcosa e' successo comunque
```

Qui non si può sostituire l'espressione con «il suo valore» perché il valore non è tutto ciò che l'espressione fa.

Su questo torniamo in L12. Da oggi a L11 lavoriamo sul funzionale, dove l'equivalenza vale.

I tipi: nessuno li dichiara, ci sono tutti

```
let doppio x = 2 * x
```

Il compilatore ricava da solo `int -> int`: poiché `*` lavora su interi, ne consegue che `x` è un intero e che anche il risultato è un intero.

Il tipo inferito è la **prima specifica** che incontrate: non l'avete scritta voi, e vincola già che cosa la funzione può fare.

Esercizi: prevedete il tipo

Nessuna annotazione. Scrivete il tipo prima di chiedere al calcolatore.

```
let a x = x
let b x y = x
let c f x = f (f x)
let d f g x = f (g x)
let h f = f 0
```

Valutazione di espressioni

- Consideriamo la funzione $f(x, y) = x + 2y$, da valutare per $x = 3$ fissato) e y arbitrario (ovvero y può variare).
- Con la lettura di OCaml, scriviamo: $f(x)(y)|_{x=3} \to f(3)(y) = g(y) = 3 + 2y$

Se vogliamo poi calcolare $g(g(1))$ il calcolo **matematico/formale** procede correttamente nel modo seguente:

$$
\begin{aligned}
g\,(g\,y) &\longrightarrow 3 + 2(g(y)) \\
&\longrightarrow 3 + 2(3 + 2 * y) \\
&\longrightarrow 3 + 6 + 4y \\
&\longrightarrow 9 + 4y|_{y=1} \\
&\longrightarrow 9 + 4 \\
&\longrightarrow \boxed{13}
\end{aligned}
$$

6 passi

OCaml procede per riduzioni

I due ingredienti delle **riduzioni**

Valore — un'espressione che non si riduce più: interi, booleani, `fun x -> ...`

Redex — un pezzo riducibile: un operatore applicato a valori, o una funzione applicata a un valore

Ridurre significa: trovare un redex, sostituirlo col suo risultato, ripetere fino a ottenere un valore.

La regola di OCaml è la riduzione per valore

Per ridurre un'applicazione `e1 e2`, OCaml:

1. riduce e1 ed e2 fino a valori;
2. solo allora effettua la sostituzione.

L'argomento si valuta sempre **prima** di entrare nella funzione.

senza effetti l'ordine non cambia il risultato, ed è di nuovo la trasparenza referenziale; con gli effetti l'ordine diventa visibile, e ci si torna a L12. Può sembrare ovvio, ma ovvio non è (come vedremo fra pochissimo).

Esercizio di comprensione

Sappiamo che la "lettura matematica" della seguente espressione:

```
let f = fun x -> fun y -> x + 2 * y in
        let g = f 3 in g (g 1)
```

produce il valore 3.

Ma come viene calcolata da OCaml?

Esercizio di comprensione

Ricordiamo preliminarmente che `f 3` è **già un valore**

```
let f = fun x -> fun y -> x + 2 * y
let g = f 3
```

- Nel body della `f` ci sono due argomenti

- Scrivendo `let g = f 3` **non ci siamo dimenticati un argomento**
- `g` è un valore a tutti gli effetti, ed è precisamente la funzione che aggiunge 3 al doppio del suo argomento
- La `g` si porta dietro il legame `x = 3`. Per questo si chiama *chiusura* (*closure*): è chiusa rispetto alle variabili che le servono.

Valutazione per nome di g (g 1)

Qualsiasi **lettura operativa** di questa espressione deve produrre, per essere corretta, lo stesso valore della lettura matematico/dichiarativa.

Operativa OCaml

$$
\begin{aligned}
g\,(g\,1) &\longrightarrow g\,(3+2*1)\\
&\longrightarrow g\,(3+2)\\
&\longrightarrow g\,5\\
&\longrightarrow 3+2*5\\
&\longrightarrow 3+10\\
&\longrightarrow \boxed{13}
\end{aligned}
$$

6 passi

Esiste un altra strategia di valutazione

Sostituire l'argomento non valutato, e ridurlo **solo quando serve**. Si chiama *valutazione per nome*

Operativa OCaml

$$
\begin{aligned}
g\,(g\,1) &\longrightarrow g\,(3+2*1)\\
&\longrightarrow g\,(3+2)\\
&\longrightarrow g\,5\\
&\longrightarrow 3+2*5\\
&\longrightarrow 3+10\\
&\longrightarrow \boxed{13}
\end{aligned}
$$

6 passi

Operativa per nome

$$
\begin{aligned}
g\,(g\,1) &\longrightarrow 3+2*(g\,1)\\
&\longrightarrow 3+2*(3+2*1)\\
&\longrightarrow 3+2*(3+2)\\
&\longrightarrow 3+2*5\\
&\longrightarrow 3+10\\
&\longrightarrow \boxed{13}
\end{aligned}
$$

6 passi

Quale strategia?

Sappiamo già quale strategia operativa usa OCaml, cioè **per valore**

Le due strategie, per valore e per nome, **non sono equivalenti**

Nell'esempio il risultato non cambia ma **solo perché** valgono due condizioni: l'espressione non ha effetti, e termina.

Senza di esse le due strategie possono dare esiti diversi, e parlare di «che cosa vale» diventa meno semplice.

Riduzioni da svolgere a mano

```
let es1 =
  let doppio = fun n -> n * 2 in
  let somma = fun a -> fun b -> a + b in
  somma (doppio 3) (doppio (doppio 1))

let es2 =
  let somma = fun x -> fun y -> x + y in
  let aggiungi3 = somma 3 in
  let applica = fun f -> fun x -> f (f x) in
  applica aggiungi3 1

let rec loop () = loop ();
let k =
    fun x -> fun y -> x in k 1 (loop ())
```

Che cosa portate a casa

- `let` lega, non assegna — e il legame viene catturato
- Il tipo c'è anche quando non lo scrivete, ed è già una specifica
- Ridurre è: trovare il redex, sostituire, ripetere
- OCaml valuta **l'argomento per primo**
- Un'espressione ha un valore *e* un percorso per arrivarci, e non sono la stessa cosa

Da fare prima di L04

In `codice/`:

1. `tipi_quiz.ml` — completate le previsioni rimaste, poi verificate in `utop`
2. `riduzioni.ml` — tre espressioni da ridurre a mano, il risultato è in fondo al file: usatelo per controllarvi, non per saltare i passi

Scrivere i passi conta più di indovinare il risultato: all'esame si valutano i passi.